

➤ What is Data Type

- A Data Type tells Python what kind of data is stored in a variable.
- They are broadly divided into two categories.
 - 1) Primitive Data Types (Basic Types)
 - 2) Non-Primitive Data Types (Collections)

1. Primitive Data Types (Basic Types)

- These are the fundamental building blocks in Python. They store single values and are mostly immutable.
 - 1) int → integers (e.g., 10, -5)
 - 2) float → decimal numbers (e.g., 3.14, -2.5)
 - 3) complex → numbers with real + imaginary parts (e.g., 2 + 3j)
 - 4) bool → Boolean values (True, False)
 - 5) str → text strings (e.g., "Hello")
 - 6) NoneType → represents "no value" (None)

```
num = 10          # int
percentage = 78.14 # float
IsPresent = True  # bool
name = "Mohit"    # str
nothing = None    # NoneType
comp_num = 2+3j   # complex
```

2. Non-Primitive Data Types (Collections)

- These are used to store multiple values or structured data. Most are mutable, except tuples.
 - 1) list → Ordered, mutable collection (e.g., [1, 2, 3])
 - 2) tuple → Ordered, immutable collection (e.g., (1, 2, 3))
 - 3) dict → Key-value pairs (e.g., {"name": "Harshal", "age": 21})
 - 4) set → Unordered collection of unique values (e.g., {1, 2, 3})

```
numbers = [1, 2, 3]          # list
coordinates = (10.0, 20.0)   # tuple
unique = {1, 2, 3, 4}        # set
person = {"name": "Mohit", "age": 30} # dict
```

3. What is Mutability

1) Mutable

- Mutable means the Variable value can be changed after it is created.
 - list → [1, 2, 3] → you can append, remove, or change elements.
 - dict → {"name": "Harshal", "age": 21} → you can add or update keys.
 - set → {1, 2, 3} → you can add or remove elements.

2) Immutable

- Immutable means the object cannot be changed once created
- If any “change” actually creates a new object in memory.
 - int → 10
 - float → 3.14
 - complex → 2 + 3j
 - bool → True
 - str → "Hello"
 - tuple → (1, 2, 3)

4. What is Type Conversion

- Type conversion means changing the data type of a value into another type. For example, turning a string "10" into an integer 10.
- Python supports two types of **type conversion** :

1) Implicit Type Conversion (Type Casting by Python)

2) Explicit Type Conversion (Type Casting by User)

1. Implicit Type Conversion (Type Casting by Python)

- Done automatically by Python.
- Python converts the smaller type into a bigger type to avoid data loss.
- Ex

```
num_int = 10      # int
num_float = 3.5   # float

result = num_int + num_float
print(result)     # 13.5 (float)
print(type(result)) # <class 'float'>
```

2. Explicit Type Conversion (Type Casting by User)

- Done manually using built-in functions like `int()`, `float()`, `str()`, `list()`, etc.
- You decide how to convert the value.
- Ex

```
# String to int
num_str = "100"
num_int = int(num_str)
print(num_int + 50)    # 150

# Float to int
pi = 3.14159
print(int(pi))         # 3

# Int to string
age = 21
print("Age is " + str(age))  # Age is 21
```

5. built-in function use with data type

1) Type()

- Returns the data type of the object.
- Helps you check what type a variable is.

```
x = 10
print(type(x))    # <class 'int'>

y = "Hello"
print(type(y))    # <class 'str'>
```

2) Id()

- Returns the unique identity number (memory address) of an variable.
- If two variables point to the same object, they will have the same id.

```
a = 5
b = a
print(id(a))    # same id as b
print(id(b))    # because integers are immutable and reused

c = [1, 2, 3]
d = [1, 2, 3]
print(id(c))    # different id
print(id(d))    # because lists are mutable and stored separately
```